



Développement d'un système de surveillance intelligent basé sur l'intelligence artificielle

Réalisé par :

Cherif Ones

Encadré par :

M. David Bomberg

2025–2026

Introduction.....	3
Objectifs.....	3
I. Développement de modèles de détection basés sur l'apprentissage supervisé.....	4
I.1 Constitution et préparation du dataset.....	4
I.1.1 Collecte des données.....	4
I.1.2 Structuration du dataset.....	4
I.2 Modélisation et entraînement des modèles.....	5
I.2.1 HOG + SVM.....	5
I.2.2 Modèle CNN.....	6
I.2.3 Modèle Random Forest.....	6
I.3 Analyse comparative des modèles.....	7
I.4 Conclusion.....	8
II. Détection de présence humaine avec modèles pré-entraînés.....	8
II.1 Présentation des modèles de détection.....	8
II.2 Analyse comparative des modèles de détection.....	9
II.3 Choix du modèle.....	9
II.4 Conclusion.....	10
III. Système de reconnaissance faciale.....	10
III.1 Architecture du système.....	10
III.2 Détection de visage.....	11
III.2.1 ROI.....	11
III.2.2 Comparaison des détecteurs de visage.....	11
III.2.3 Validation en conditions réelles.....	11
III.2.4 Validation du pipeline YOLO + DNN.....	12
III.3 Reconnaissance faciale.....	13
III.3.1 Principe.....	13
III.3.2 Comparaison des modèles.....	13
III.4 Résultats du pipeline complet.....	14
III.5 Conclusion.....	15
IV. Interface web de supervision.....	15
IV.1 Architecture technique.....	15
IV.2 Communication Raspberry Pi → Backend.....	16
IV.3 Fonctionnalités.....	16
V.Limites & perspectives.....	19
Limites.....	19
Malgré les résultats obtenus, le système présente plusieurs limites identifiées :.....	19
• Qualité d'image à distance : lorsque la personne est éloignée de la caméra, le visage extrait par le DNN est de faible résolution, ce qui dégrade la précision de FaceNet512. Une étape de super-résolution pourrait améliorer ce point, mais son coût computationnel est incompatible avec les ressources de la Pi 4 (CPU à 350%, RAM à 75%).....	19
• Sensibilité aux conditions d'acquisition : la reconnaissance faciale reste sensible aux variations importantes de luminosité et d'angle, pouvant générer des faux négatifs.....	19

• FPS variable : le FPS du pipeline dépend du nombre de personnes détectées — chaque personne supplémentaire nécessite un passage DNN + FaceNet512, réduisant les performances globales.....	19
• Absence de sécurisation : les communications entre la Pi, le backend et le frontend transitent en HTTP non chiffré, sans authentification.....	19
Perspectives.....	19
• Intégrer un accélérateur matériel Coral TPU pour délester le CPU des calculs IA et améliorer significativement le FPS.....	19
• Ajouter une étape de super-résolution entre le DNN et FaceNet512 pour améliorer la reconnaissance à distance.....	19
• Développer une application mobile avec notifications push en temps réel.....	19
• Sécuriser les communications avec HTTPS et authentification JWT sur les endpoints FastAPI.....	19
Conclusion générale :.....	19
Diagramme de Gant.....	20

Introduction

Dans un contexte où les systèmes de surveillance intelligents occupent une place croissante dans la sécurité des espaces physiques, ce projet vise à concevoir et développer un système embarqué de détection de présence humaine et de reconnaissance faciale en temps réel. Le système repose sur une Raspberry Pi 4 comme plateforme de traitement, en exploitant les capacités de l'intelligence artificielle embarquée pour analyser le flux vidéo directement sur le dispositif.

Objectifs

Les objectifs de ce projet sont :

- Développer et comparer des modèles de détection de présence humaine basés sur l'apprentissage supervisé
- Identifier et optimiser un pipeline de détection temps réel adapté aux contraintes d'une plateforme embarquée

- Intégrer un système de reconnaissance faciale permettant d'identifier les personnes enregistrées
- Concevoir une interface web de supervision pour la gestion de la base de données et la visualisation des détections.

I. Développement de modèles de détection basés sur l'apprentissage supervisé

Une première approche a consisté à développer des modèles de détection de présence humaine à partir d'un jeu de données constitué spécifiquement pour ce projet. L'objectif est d'évaluer les performances de ces modèles en termes de précision et de coût computationnel, afin d'identifier la solution la plus adaptée à un déploiement sur plateforme embarquée

I.1 Constitution et préparation du dataset

I.1.1 Collecte des données

Un ensemble d'images a été constitué afin de représenter deux classes distinctes :

- la présence humaine (classe 1)
- l'absence de présence humaine (classe 0).

Les données ont été collectées à partir de différentes sources en ligne, incluant des images extraites de vidéos de surveillance.

Le jeu de données final est composé de 7 904 images, réparties comme suit :

- 1 700 images pour la classe 0 (absence de présence humaine) ;
- 6 204 images pour la classe 1 (présence humaine).

I.1.2 Structuration du dataset

Le dataset a été organisé en trois sous-ensembles distincts :

- Ensemble d'apprentissage (train), utilisé pour entraîner les modèles ;
- Ensemble de validation (validation), utilisé pour ajuster les paramètres et surveiller le surapprentissage ;
- Ensemble de test (test), utilisé uniquement pour l'évaluation finale des performances.

La répartition des données est résumée dans le tableau suivant :

Ensemble	Classe 0	Classe 1
Apprentissage	1190	1190
Validation	255	931
Test	1700	930

L'ensemble d'apprentissage a été équilibré afin d'éviter un biais du modèle en faveur de la classe majoritaire lors de l'entraînement.

En revanche, les ensembles de validation et de test conservent la distribution réelle des données, ce qui permet une évaluation plus représentative des performances en conditions réelles.

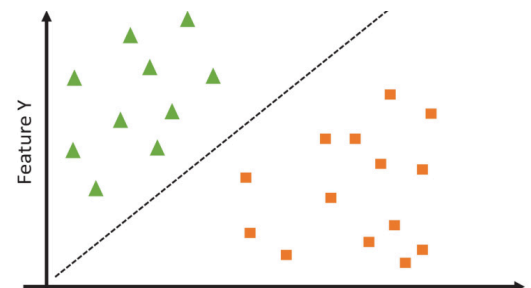
I.2 Modélisation et entraînement des modèles

Trois modèles ont été développés et comparés : HOG + SVM, CNN et Random Forest. Chacun repose sur un paradigme différent, du traitement classique de l'image à l'apprentissage profond.

I.2.1 HOG + SVM

Principe

Ce modèle repose sur l'extraction de descripteurs HOG (Histogram of Oriented Gradients), qui capturent les variations locales de gradients caractéristiques des silhouettes humaines. Ces descripteurs alimentent ensuite un classifieur SVM linéaire qui sépare les deux classes en maximisant la marge entre elles. Les images ont été redimensionnées en 128×128 pixels et converties en niveaux de gris avant extraction.



Implémentation

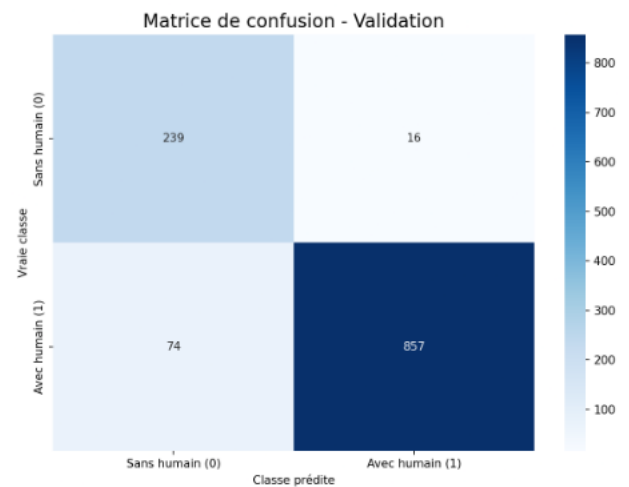
Les paramètres HOG retenus sont : 9 orientations, 8×8 pixels par cellule et 2×2 cellules par bloc. L'entraînement a été réalisé sur PC.

Résultats

Les performances du modèle ont été évaluées à l'aide d'une matrice de confusion :

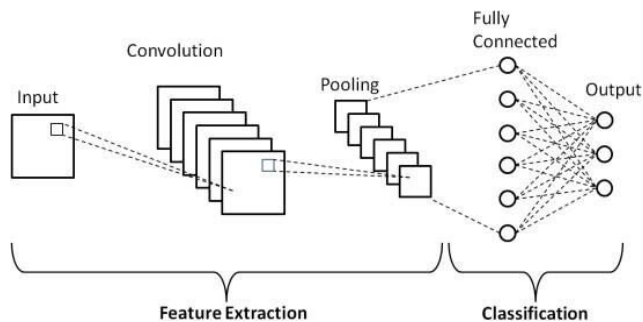
- 239 vrais négatifs
- 857 vrais positifs
- 16 faux positifs
- 74 faux négatifs.

=> Le modèle présente un nombre de faux négatifs non négligeable : certaines personnes présentes dans les images ne sont pas détectées, ce qui constitue une limitation dans un contexte de surveillance.



I.2.2 Modèle CNN

Principe



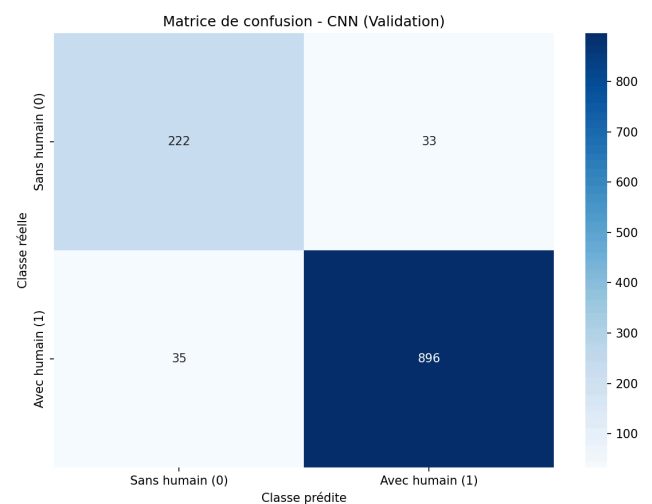
Contrairement à HOG + SVM, le CNN (Convolutional Neural Network) apprend automatiquement les caractéristiques pertinentes directement à partir des images, sans étape d'ingénierie de features. Il extrait des motifs visuels hiérarchiques (contours, formes, structures) à travers ses couches de convolutions successives.

Implémentation

Le modèle a été entraîné sur PC avec les hyper paramètres suivants : 20 epochs, batch size de 32, optimiseur Adam avec un learning rate de 0.001.

Résultats

- 222 vrais négatifs
- 896 vrais positifs
- 33 faux positifs
- 35 faux négatifs

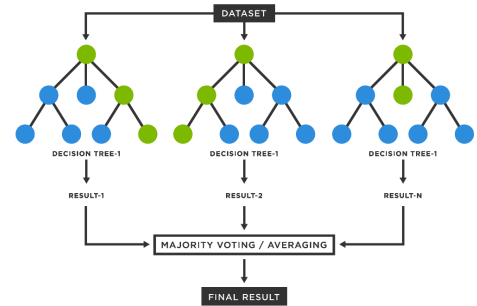


=> Le CNN réduit significativement les faux négatifs (35 contre 74 pour le SVM), traduisant une meilleure capacité à détecter les personnes présentes. Cette amélioration s'accompagne d'une légère hausse des faux positifs, mais le modèle offre globalement la détection la plus équilibrée des trois

I.2.3 Modèle Random Forest

Principe

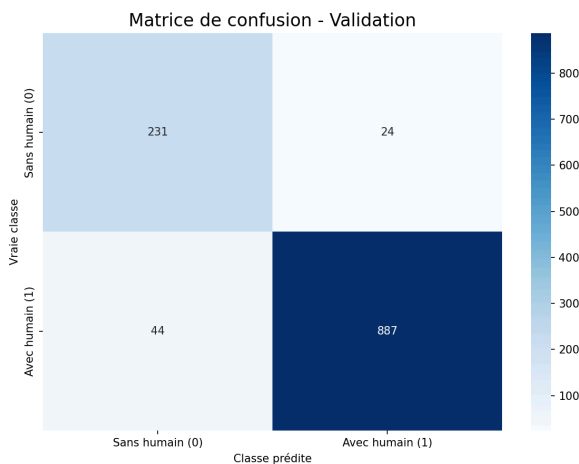
Le Random Forest est une méthode d'ensemble supervisée basée sur un ensemble d'arbres de décision. Chaque arbre est entraîné sur un sous-ensemble aléatoire des données, et la prédiction finale est obtenue par vote majoritaire entre les arbres.



Implémentation

Le modèle a été entraîné sur PC avec 300 arbres et une profondeur maximale de 20. Les descripteurs HOG extraits servent d'entrée à chaque arbre.

Résultats



- 231 vrais négatifs
- 887 vrais positifs
- 24 faux positifs
- 44 faux négatif

=> Le Random Forest offre un compromis intermédiaire : plus précis que le SVM en termes de faux négatifs, mais sans amélioration significative par rapport au CNN. Reposant sur des descripteurs HOG, il hérite des mêmes limitations liées à la représentation des caractéristiques

I.3 Analyse comparative des modèles

Le tableau suivant synthétise les performances des trois modèles en termes de recall (la capacité du modèle à détecter les personnes présentes), F1-score (mesure globale équilibrée des performances) et temps de traitement par image.

Modèle	Recal	F1-score	Temps (s)
SVM+HOG	0.92	0.95	0.0138s
CNN	0.96	0.96	0.0857s
RF	0.95	0.96	0.0820

Le SVM se distingue par sa rapidité mais présente le recall le plus faible. Le CNN obtient les meilleures performances de détection avec un recall de 0.96, au prix d'un temps de traitement plus élevé. Le Random Forest se situe entre les deux, sans apporter d'amélioration notable par rapport au CNN.

I.4 Conclusion

Les trois modèles développés présentent des performances satisfaisantes dans le cadre expérimental contrôlé du dataset. Cependant, lors de tests sur des scènes réelles — avec des variations d'éclairage, d'angles de vue et d'environnements — une dégradation notable des performances a été observée sur l'ensemble des modèles.

Cette limitation s'explique par le manque de diversité du dataset d'entraînement : les images collectées proviennent majoritairement de contextes similaires (environnements intérieurs, angles fixes, conditions d'éclairage stables), ce qui empêche les modèles d'apprendre des représentations suffisamment généralisables.

Une solution aurait été d'enrichir le dataset avec des images plus variées, mais cette approche reste coûteuse en temps.

Par conséquent, afin de poursuivre efficacement le développement du système tout en respectant les contraintes de temps, le choix s'est orienté vers l'utilisation de modèles pré-entraînés. Ces modèles, entraînés sur des bases de données à grande échelle et fortement diversifiées (plusieurs centaines de milliers à plusieurs millions d'images), permettent d'obtenir une meilleure robustesse face à des scènes complexes et variées.

II. Détection de présence humaine avec modèles pré-entraînés

Face aux limites de généralisation des modèles supervisés développés en partie I, trois approches basées sur des modèles pré-entraînés ont été étudiées : HOG, SSD et YOLO.

II.1 Présentation des modèles de détection

Détection basée sur HOG : méthode classique de vision par ordinateur basée sur l'analyse des gradients de l'image. Les descripteurs extraits sont comparés à un modèle pré-entraîné intégré dans OpenCV.

Détection basée sur SSD : réseau convolutif qui détecte les objets en une seule passe, prédisant simultanément les bounding boxes et les classes. Plus robuste que HOG grâce à l'apprentissage automatique des caractéristiques

Détection basée sur YOLO : modèle de détection en temps réel qui traite l'image en une seule passe et prédit simultanément positions et classes. Optimisé pour la rapidité tout en conservant une précision élevée.

Tous les modèles suivent une structure globale similaire : une image est prétraitée (redimensionner) , puis analysée afin de détecter la présence d'une personne.

- Les modèles basés sur le deep learning (SSD et YOLO) produisent un score de confiance pour chaque détection, qui est ensuite comparé à un seuil.
- Le modèle HOG extrait des features et les donne à un SVM qui décide si c'est une personne ou non (inclus dans la bibliothèque OpenCv).
- Le modèle retenu est YOLOv8n (variante nano), le plus léger de la famille YOLOv8, particulièrement adapté aux contraintes matérielles de la Raspberry Pi 4.

II.2 Analyse comparative des modèles de détection

Les performances ont été comparées selon trois critères principaux : le temps de traitement, le nombre d'images traitées par seconde (FPS) et la précision de détection.

Les résultats moyennés obtenus sont résumés dans le tableau suivant :

Modèle	Temps (s)	FPS	Précision
HOG	0.1	~10 FPS	Très faible
SSD	0.3	~3 FPS	moyenne
YOLO	0.65	~1.64 FPS	Très élevée

HOG présente le temps de traitement le plus faible mais échoue dans des conditions complexes (variations d'éclairage, angles de vue chargés), le rendant inadapté à la surveillance. SSD constitue une solution intermédiaire, plus robuste que HOG mais moins performant que YOLO. Ce dernier offre les meilleures performances en précision et en généralisation, malgré un temps de traitement plus élevé.

II.3 Choix du modele

Après plusieurs tests réalisés dans des conditions variées (angles de prise de vue, distances, éclairages), YOLO a été retenu pour sa fiabilité de détection supérieure. Des optimisations ont ensuite été recherchées afin de maintenir cette précision tout en réduisant le temps de traitement sur la Raspberry Pi 4.

- Réduction de la résolution d'entrée : après plusieurs tests, une taille de 320×320 pixels a été retenue comme meilleur compromis entre précision et performance, réduisant le temps d'inférence d'environ 0.15 s par image.
- Frame skipping : seule une image sur N est traitée par le pipeline, permettant d'augmenter le FPS global du système sans dégrader significativement la qualité de détection.

- Export du modèle au format ONNX : le processeur ARM Cortex-A72 de la Raspberry Pi 4 intègre une unité de calcul vectoriel appelée ARM NEON, capable d'effectuer plusieurs opérations mathématiques en parallèle en une seule instruction — ce qui est particulièrement adapté aux calculs matriciels intensifs des réseaux de neurones. Cependant, PyTorch ne détecte pas automatiquement cette unité sur architecture ARM et n'en tire pas parti, limitant les performances à ~ 0.6 FPS. Pour contourner cette limitation, le modèle YOLOv8n a été converti du format natif PyTorch (.pt) vers le format ONNX (.onnx). ONNX Runtime détecte automatiquement la présence de ARM NEON et l'exploite nativement pour paralléliser les calculs, réduisant le temps d'inférence d'environ $9\times$ et faisant passer le FPS de ~ 0.6 à ~ 5.5 .

II.4 Conclusion

Les optimisations mises en place ont permis de rendre YOLO viable sur la Raspberry Pi 4 tout en conservant une précision de détection satisfaisante dans des conditions réelles.

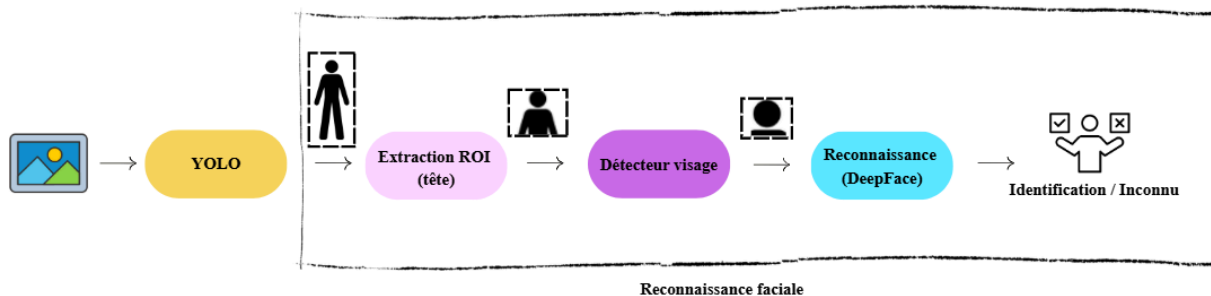
Néanmoins, YOLO se limite à localiser une personne dans la scène sans pouvoir l'identifier. Pour répondre pleinement à l'objectif du projet, le pipeline a été étendu vers la reconnaissance faciale, ce qui fait l'objet de la partie suivante

III. Système de reconnaissance faciale

La détection de présence humaine étant maîtrisée, le système a été étendu vers une fonctionnalité d'identification : reconnaître non seulement qu'une personne est présente, mais également qui elle est. Cette partie présente l'architecture du système de reconnaissance faciale développé et intégré au pipeline existant.

III.1 Architecture du système

Le système développé repose sur une chaîne de traitement permettant de détecter la présence d'une personne puis d'identifier son visage en temps réel. Le fonctionnement global s'articule en quatre étapes successives :



1. Une image issue de la caméra est analysée par YOLO qui fournit des zones de détection (bounding boxes) correspondant aux individus présents.

2. Pour chaque personne détectée, une région d'intérêt (ROI) est extraite en se concentrant sur une partie de la box.
3. Cette région du Roi est traitée par un détecteur de visage. Cette étape permet d'extraire précisément le visage.
4. Une fois le visage extrait, celui-ci est transmis à un module de reconnaissance faciale du deepface. Ce module convertit le visage en un vecteur de caractéristiques (embedding), qui est ensuite comparé à une base de données de visages connus afin d'identifier la personne

III.2 Détection de visage

III.2.1 ROI

Pour chaque personne détectée par YOLO, seule la partie supérieure de la bounding box est conservée, correspondant à environ 40 % de la hauteur totale. Cette optimisation est particulièrement utile lorsque YOLO détecte le corps entier de la personne — en ne transmettant que la région du visage au détecteur DNN, on réduit significativement le nombre de pixels à analyser, allégeant ainsi la charge de traitement du pipeline. Cependant, lorsque la personne est proche de la caméra et que la bounding box est déjà serrée autour du visage, cette découpe peut risquer de tronquer le visage. Le crop YOLO est donc utilisé directement dans ce cas.

III.2.2 Comparaison des détecteurs de visage

Plusieurs modèles de détection de visage ont été évalués afin d'identifier la solution la plus adaptée au système. Les modèles testés sont Haar Cascade, HOG, MTCNN et le modèle DNN d'OpenCV. Les tests ont été réalisés sur une image issue de la caméra contenant un visage.

Modèle	Visages détectés	Temps
haar_frontal_default	3	~1.36 s
haar_frontal_alt2	1	~1.49 s
haar_frontal_alt	1	~1.72 s
haar_profile	1	~2.35 s
haar_frontal_alt_tree	0	~1.12 s
hog_face_recognition	1	~1.59s
opencv_dnn_ssd	1	~0.22s
mtcnn	1	~1.63s

haar_frontal_alt_tree n'a détecté aucun visage et haar_frontal_default en a détecté trois, incluant des fausses détections — ces deux variantes ont donc été écartées. Les autres modèles détectent correctement un visage. Afin de départager les modèles restants, plusieurs tests ont été menés sur des images variées (angles, éclairages, distances) afin d'identifier le plus robuste.

III.2.3 Validation en conditions réelles

Afin de confirmer le choix du détecteur de visage dans le contexte du pipeline complet, plusieurs combinaisons YOLO + détecteur ont été testées sur des images issues de la caméra, après extraction de la ROI.

Remarque :

Ces mesures ont été réalisées avec le modèle YOLOv8n au format PyTorch (.pt). L'optimisation ONNX présentée en II.3 a permis par la suite d'améliorer ces performances d'un facteur ~8.

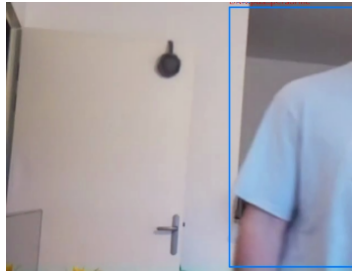
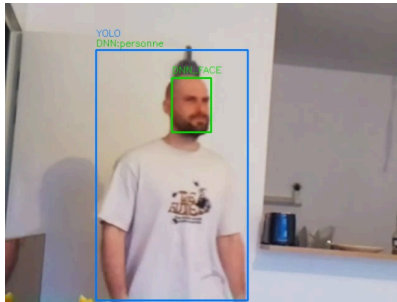
Détecteur de visage + yolo	temps
YOLO	0.496
YOLO + HAAR	1.052s
YOLO + DNN	1.1130
YOLO + MTCNN	1.9888

HAAR est légèrement plus rapide que DNN mais montre des échecs de détection lorsque l'angle du visage varie. MTCNN, bien que précis, est trop lent pour un usage temps réel sur Pi. Le DNN offre le meilleur compromis entre précision et temps de traitement, ce qui confirme le choix de la combinaison **YOLO + DNN**.

III.2.4 Validation du pipeline YOLO + DNN

Le pipeline YOLO + DNN a été évalué sur une vidéo de test de 212 frames afin de valider ses performances avant l'intégration de la reconnaissance faciale.

Métrique	Valeur
Frames totales	212
Frames avec personne détectée	131 (~62 %)
Visages détectés	73 (~56 % des personnes)
Temps total YOLO	77.48 s (~70 %)
Temps total DNN	32.06 s (~30 %)
Temps total pipeline	110.92 s
Temps moyen par frame	~0.52 s (~1.9 FPS)



L'écart entre le nombre de personnes détectées (131) et le nombre de visages détectés (73) s'explique par le fait que YOLO peut détecter une personne même lorsque son visage n'est pas visible — personne de dos, visage hors cadre, etc.

YOLO représente 70 % du temps de traitement total. C'est pourquoi l'optimisation du modèle au format ONNX présentée en II.3 a été prioritaire, permettant de réduire significativement ce temps d'inférence.

III.3 Reconnaissance faciale

III.3.1 Principe

La reconnaissance faciale est assurée par la bibliothèque DeepFace, qui fournit une interface unifiée pour plusieurs modèles de reconnaissance. Le principe repose sur la génération d'un embedding — vecteur de caractéristiques représentant le visage — qui est ensuite comparé aux embeddings de la base de données via une distance cosinus. Une personne est reconnue si la distance est inférieure à un seuil fix..

III.3.2 Comparaison des modèles

Afin d'évaluer les performances des modèles de reconnaissance faciale, une campagne de tests a été réalisée sur la Raspberry Pi 4. Plusieurs images de référence correspondant à différentes personnes ont été utilisées, et une image issue du flux caméra (après détection et extraction du visage via le DNN) a été comparée à l'ensemble des visages enregistrés, simulant ainsi un cas réel d'identification.

Chaque modèle a été exécuté séparément afin d'éviter les biais liés à la charge cumulée. Les paramètres suivants ont été conservés pour tous les tests : même image de référence, même ensemble de les plusieurs images test, métrique cosinus, backend de détection opencv.

Modèle	Matches trouvés	Temps moyen (s)
VGG-Face	1	22.89
Facenet	0	17.36

Facenet512	1	17.01
ArcFace	1	15.36
Dlib	1	9.48

Dlib est le plus rapide avec un temps moyen inférieur à 10 s, mais sa précision reste limitée dans des conditions variées. Facenet n'a trouvé aucune correspondance dans ce protocole, ce qui le disqualifie directement pour ce système. VGG-Face est le plus coûteux en temps (~23 s) sans offrir d'avantage notable en précision. ArcFace offre de bonnes performances mais des tests complémentaires en conditions réelles ont révélé des confusions entre personnes proches, limitant sa fiabilité dans ce contexte.

Facenet512 a été retenu comme modèle final. Générant des embeddings de dimension 512, il offre une représentation riche et discriminante des visages. Avec un temps moyen de 17.01 s sur Pi et une meilleure robustesse face aux variations d'apparence (éclairage, angles, expressions), il constitue le meilleur compromis entre précision et coût computationnel dans ce contexte embarqué.

III.4 Résultats du pipeline complet

Le pipeline complet — intégrant YOLOv8n (ONNX), le détecteur DNN OpenCV et Facenet512 — a été évalué sur un flux vidéo en conditions réelles sur la Raspberry Pi 4. Les résultats obtenus sont les suivants :

Métrique	Valeur
Frames traitées	1642
FPS pipeline complet	2.78 FPS
FPS YOLO seul	5.77 FPS
RAM	75%
CPU	350%

Le pipeline atteint 2.78 FPS en conditions réelles, ce qui est suffisant pour une application de surveillance où les mouvements sont lents par rapport à la fréquence d'acquisition. YOLO seul atteint 5.77 FPS, confirmant que Facenet512 constitue la partie la plus coûteuse du pipeline. Ces performances sont obtenues sans GPU, uniquement sur le CPU ARM Cortex-A72 de la Raspberry Pi 4.

Base de données de référence : Pour chaque personne à reconnaître, un dossier dédié est créé contenant plusieurs photos prises sous différents angles (face, profil, trois-quarts) et dans des conditions d'éclairage

variées. Au lancement du pipeline, Facenet512 calcule les embeddings de toutes ces images de référence et les conserve en mémoire. Cette étape est réalisée une seule fois au démarrage.

Afin d'éviter les envois redondants, un mécanisme anti-spam a été mis en place : tant que la personne détectée est identique à la dernière détection, aucun nouvel envoi n'est effectué. Une nouvelle notification n'est déclenchée que lorsqu'un changement de personne est détecté. Ce mécanisme gère également le cas où une personne quitte le champ de la caméra — lorsqu'aucune personne n'est détectée, la variable de dernière détection est réinitialisée à None. Ainsi, si la même personne revient dans le champ, une nouvelle notification est bien déclenchée car son nom diffère de None."

```
[MATCH] Ones | dist=0.3789
[MATCH] Inconnu | dist=0.4477
```

III.5 Conclusion

Le système de reconnaissance faciale développé repose sur une chaîne de traitement complète et optimisée pour la Raspberry Pi 4. Le choix de chaque composant a été guidé par des tests rigoureux : YOLOv8n (ONNX) pour la détection de personnes, le DNN OpenCV pour l'extraction du visage, et Facenet512 pour la reconnaissance.

Ce pipeline constitue le cœur du système. Il sera complété dans la partie suivante par une interface web de supervision permettant de gérer la base de données et de visualiser les détections en temps réel afin de faciliter l'expérience utilisateur.

IV. Interface web de supervision

Fort du pipeline de détection et reconnaissance développé sur la Raspberry Pi 4, une interface web a été conçue pour permettre la supervision du système en temps réel et la gestion de la base de données des personnes enregistrées.

IV.1 Architecture technique

L'application repose sur une architecture client-serveur :

Composant	Technologie	Rôle
Frontend	React + Vite	Interface utilisateur
Backend	FastAPI (Python)	API REST, logique métier
Base de données	SQLite	Historique des détections et personnes

Serveur Pi	FastAPI (Python)	Gestion du dossier BD
------------	------------------	-----------------------

Choix technologiques :

- **React** a été retenu pour sa gestion efficace des mises à jour d'interface — le dashboard se rafraîchit automatiquement toutes les 3 secondes sans rechargement de page, ce qui est essentiel pour un affichage temps réel.
- **FastAPI** a été choisi pour sa rapidité d'exécution, sa génération automatique de documentation interactive (/docs) qui permet de tester facilement les API et sa syntaxe moderne basée sur les annotations Python.
- **SQLite** a été choisi pour sa légèreté — aucune installation de serveur de base de données n'est nécessaire, le fichier database.db est créé automatiquement au démarrage.

IV.2 Communication Raspberry Pi → Backend

Lorsque le pipeline détecte une personne, la Raspberry Pi envoie automatiquement une requête HTTP POST au backend FastAPI avec les informations suivantes : nom de la personne, distance cosinus et statut (reconnu / inconnu). L'heure et la date sont enregistrées automatiquement côté serveur.

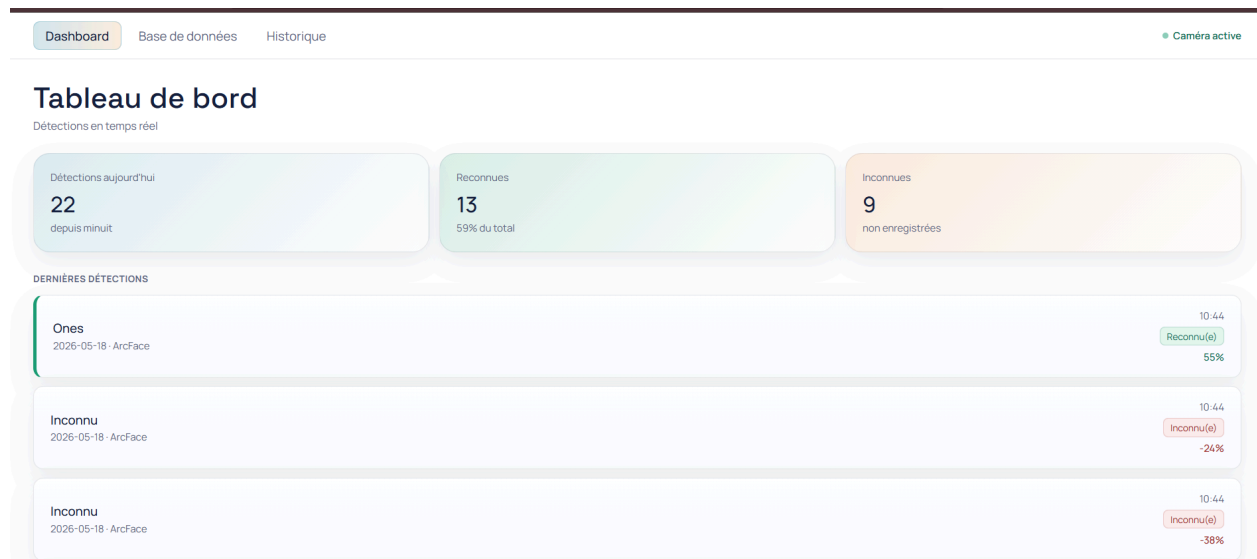
Le backend expose les routes suivantes :

Méthode	Route	Description
POST	/detections	Enregistre une nouvelle détection
GET	/detections	Retourne l'historique complet
GET	/personnes	Retourne la liste des personnes
POST	/personnes	Ajoute une personne avec sa photo
DELETE	/personnes/{nom}	Supprime une personne

IV.3 Fonctionnalités

L'interface comprend trois pages :

Dashboard — page principale affichant les détections en temps réel. Trois métriques sont affichées en haut de page : nombre total de détections du jour, nombre de personnes reconnues et nombre d'inconnus. Chaque détection apparaît sous forme de carte avec le nom, l'heure, le statut (reconnu/inconnu) et le pourcentage de certitude. La page se rafraîchit automatiquement toutes les 3 secondes.



La distance cosinus produite par Facenet512 est une valeur entre 0 et 1, peu intuitive pour un utilisateur non technique. Elle est donc convertie en pourcentage de certitude selon deux échelles distinctes, selon que la personne est reconnue ou non.

Pour une personne reconnue ($\text{distance} \in [0, 0.5]$) $\rightarrow \text{certitude} = (1 - \text{distance} / 0.50) \times 100$

Pour une personne inconnue ($\text{distance} > 0.5$) $\rightarrow \text{certitude} = ((\text{distance} - 0.50) / 0.50) \times 100$

Base de données — gestion des personnes enregistrées — ajout et suppression. Lors de l'ajout d'une personne, deux modes sont disponibles pour fournir les photos de référence : l'upload de fichiers images depuis l'appareil, ou la capture directe depuis la caméra du PC ou du téléphone via le navigateur. Ce second mode permet de prendre plusieurs photos sous différents angles sans quitter l'interface, enrichissant ainsi la base de référence. L'ajout d'une personne déclenche automatiquement l'envoi des photos vers la Raspberry Pi.

NOUVELLE PERSONNE

Nom complet

ex: Alice Martin

Photos de référence

Fichiers

Caméra



Capturer

0 photo prise

Annuler

Enregistrer (0 photo)

Historique — page de consultation de toutes les détections passées, avec filtres par période (aujourd'hui, 7 derniers jours, tout) et par personne. Les résultats sont affichés sous forme de tableau groupé par date.

DashboardBase de donnéesHistorique

Caméra active

Historique

Toutes les détections passées

Aujourd'hui

7 derniers jours

Tout

AUJOURD'HUI – 2026-05-26

PERSONNE	HEURE	STATUT	CERTITUDE
Amani	20:26	Reconnu(e)	38%
Tanguy	20:25	Reconnu(e)	60%
Inconnu	13:53	Inconnu(e)	0%
Ones	13:53	Reconnu(e)	41%
Ones	13:53	Reconnu(e)	41%

V.Limites & perspectives

Limites

Malgré les résultats obtenus, le système présente plusieurs limites identifiées :

- Qualité d'image à distance : lorsque la personne est éloignée de la caméra, le visage extrait par le DNN est de faible résolution, ce qui dégrade la précision de FaceNet512. Une étape de super-résolution pourrait améliorer ce point, mais son coût computationnel est incompatible avec les ressources de la Pi 4 (CPU à 350%, RAM à 75%).
- Sensibilité aux conditions d'acquisition : la reconnaissance faciale reste sensible aux variations importantes de luminosité et d'angle, pouvant générer des faux négatifs.
- FPS variable : le FPS du pipeline dépend du nombre de personnes détectées — chaque personne supplémentaire nécessite un passage DNN + FaceNet512, réduisant les performances globales.
- Absence de sécurisation : les communications entre la Pi, le backend et le frontend transitent en HTTP non chiffré, sans authentification.

Perspectives

- Intégrer un accélérateur matériel Coral TPU pour délester le CPU des calculs IA et améliorer significativement le FPS
- Ajouter une étape de super-résolution entre le DNN et FaceNet512 pour améliorer la reconnaissance à distance
- Développer une application mobile avec notifications push en temps réel
- Sécuriser les communications avec HTTPS et authentification JWT sur les endpoints FastAPI

Conclusion générale :

Ce projet a permis de développer un système complet de surveillance intelligent, allant de la détection de présence humaine à la reconnaissance faciale, le tout déployé sur une Raspberry Pi 4 sans GPU. Chaque composant du pipeline — YOLOv8n, DNN OpenCV et Facenet512 — a été sélectionné après une phase de tests rigoureux, en privilégiant le meilleur compromis entre précision et performance embarquée.

L'interface web développée complète le système en offrant une supervision en temps réel et une gestion intuitive de la base de données. Ce travail démontre la faisabilité d'un système de surveillance autonome et intelligent sur plateforme contrainte, ouvrant des perspectives d'amélioration notamment sur la vitesse de reconnaissance et l'enrichissement de la base de données.

Diagramme de Gantt

